

C++ Standard Library Issues to be moved in Virtual Plenary, Feb. 2021

Doc. no. P2315R0

Date: 2021-02-12

Audience: WG21

Reply to: Jonathan Wakely <lwgchair@gmail.com>

Tentatively Ready Issues

[3391](#). Problems with `counted_iterator/move_iterator::base() const &`

Section: 23.5.3 [\[move.iterators\]](#), 23.5.6 [\[iterators.counted\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Patrick Palka **Opened:** 2020-02-07
Last modified: 2020-02-10

Priority: 2

View all other [issues](#) in `[move.iterators]`.

Discussion:

It is not possible to use the `const &` overloads of `counted_iterator::base()` or `move_iterator::base()` to get at an underlying move-only iterator in order to compare it against a sentinel.

More concretely, assuming issue LWG [3389](#) is fixed, this means that

```
auto v = r | views::take(5);
ranges::begin(v) == ranges::end(v);
```

is invalid when `r` is a view whose `begin()` is a move-only input iterator. The code is invalid because `ranges::take_view::sentinel::operator==( )` must call `counted_iterator::base()` to compare the underlying iterator against its sentinel, and therefore this `operator==( )` requires that the underlying iterator is `copy_constructible`.

Suggested resolution:

Make these `const &` `base()` overloads return the underlying iterator by `const` reference. Remove the `copy_constructible` constraint on these overloads. Perhaps the `base()` overloads for the iterator wrappers in 24.7 [\[range.adaptors\]](#) could use the same treatment?

[2020-02 Prioritized as P2 Monday morning in Prague]

[2021-01-28; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4849](#).

1. Modify 23.5.3.2 [\[move.iterator\]](#), class template `move_iterator` synopsis, as indicated:

```
namespace std {
    template<class Iterator>
    class move_iterator {
    public:
        using iterator_type = Iterator;
        [...]
        constexpr const iterator_type& base() const &;
        constexpr iterator_type base() &&;
        [...]
    };
}
```

2. Modify 23.5.3.5 [\[move.iter.op.conv\]](#) as indicated:

```
constexpr const Iterator& base() const &;
```

~~1- Constraints: Iterator satisfies copy_constructible.~~

~~2- Preconditions: Iterator models copy_constructible.~~

-3- Returns: current.

3. Modify 23.5.6.1 [\[counted.iterator\]](#), class template counted_iterator synopsis, as indicated:

```
namespace std {  
    template<input_or_output_iterator I>  
    class counted_iterator {  
    public:  
        using iterator_type = I;  
        [...]   
        constexpr const I& base() const & requires copy_constructible<I>;  
        constexpr I base() &&;  
        [...]   
    };  
}
```

4. Modify 23.5.6.3 [\[counted.iter.access\]](#) as indicated:

```
constexpr const I& base() const & requires copy_constructible<I>;
```

-1- Effects: Equivalent to: return current;

3433. `subrange::advance(n)` has UB when $n < 0$

Section: 24.5.4.3 [\[range.subrange.access\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2020-04-21 **Last modified:** 2020-09-06

Priority: 2

Discussion:

`subrange::advance(n)` is specified to call `ranges::advance(begin_, n, end_)` in both *StoreSize* and *!StoreSize* cases (24.5.4.3 [\[range.subrange.access\]](#)/9). Unfortunately, `ranges::advance(begin_, n, end_)` has undefined behavior when $n < 0$ unless `[end_, begin_)` denotes a valid range (23.4.4.2 [\[range.iter.op.advance\]](#)/5). This would all be perfectly fine — the UB is exposed to the caller via effects-equivalent-to — were it not the design intent that `subrange::advance(-n)` be usable to reverse the effects of `subrange::advance(n)` when the subrange has a bidirectional iterator type. That is, however, clearly the design intent: `subrange::prev()`, for example, is equivalent to `subrange::advance(-1)`.

[2020-05-11; Reflector prioritization]

Set priority to 2 after reflector discussions.

[2021-01-15; Issue processing telecon]

Set status to Tentatively Ready after discussion and poll.

FAN

7 0 3

Proposed resolution:

This wording is relative to [N4861](#).

1. Modify 24.5.4.3 [\[range.subrange.access\]](#) as indicated:

```
constexpr subrange& advance(iterator_difference_t<I> n);
```

-9- Effects: Equivalent to:

~~(9.1) — If *StoreSize* is true,~~

```

auto d = n - ranges::advance(begin_, n, end_);
if (d >= 0)
    size_ -= to_unsigned_like(d);
else
    size_ += to_unsigned_like(-d);
return *this;

(0.2) Otherwise,

ranges::advance(begin_, n, end_);
return *this;

if constexpr (bidirectional_iterator<I>){
    if (n < 0){
        ranges::advance(begin_, n);
        if constexpr (StoreSize)
            size_ += to_unsigned_like(-n);
        return *this;
    }
}

auto d = n - ranges::advance(begin_, n, end_);
if constexpr (StoreSize)
    size_ -= to_unsigned_like(d);
return *this;

```

3490. ranges::drop_while_view::begin() is missing a precondition

Section: 24.7.10.2 [\[range.drop.while.view\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Michael Schellenberger Costa **Opened:** 2020-10-13
Last modified: 2020-11-07

Priority: 0

View all other [issues](#) in `[range.drop.while.view]`.

Discussion:

Similar to ranges::filter_view 24.7.5.2 [\[range.filter.view\]](#) p3, ranges::drop_while_view should have a precondition on its begin() method that the predicate is set.

I propose to add as 24.7.10.2 [\[range.drop.while.view\]](#) p3:

```
Preconditions: pred .has value().
```

[2020-11-07; Reflector prioritization]

Set priority to 0 and status to Tentatively Ready after six votes in favour during reflector discussions.

Proposed resolution:

This wording is relative to [N4868](#).

1. Modify 24.7.10.2 [\[range.drop.while.view\]](#) as indicated:

Since we usually don't rely on implicit bool conversion in *Preconditions*: elements an explicit "is true" has been added. Editorial fixes of the referenced paragraph 24.7.10.2 [\[range.drop.while.view\]](#) p3 and similar places have been requested separately.

```
constexpr auto begin();
```

```
-?- Preconditions: pred .has value().is true.
```

```
-3- Returns: ranges::find_if_not(base_, cref(*pred_)).
```

```
-4- [...]
```

[3492](#). Minimal improvements to `elements_view::iterator`

Section: 24.7.16.3 [\[range.elements.iterator\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Michael Schellenberger Costa **Opened:** 2020-10-28 **Last modified:** 2020-11-15

Priority: 0

View other [active issues](#) in `[range.elements.iterator]`.

View all other [issues](#) in `[range.elements.iterator]`.

Discussion:

During code review of `elements_view` for MSVC-STL we found two issues that should be easily addressed:

1. `elements_view::iterator` constraints both `operator++(int)` member functions

```
constexpr void operator++(int) requires (!forward_range<Base>);
constexpr iterator operator++(int) requires forward_range<Base>;
```

However, given that a constrained method would be preferred we only need to constrain one of those. The proposal would be to remove the constraint from the void returning overload and change the declaration to

```
constexpr void operator++(int);
constexpr iterator operator++(int) requires forward_range<Base>;
```

2. `elements_view::iterator` `operator-` is constrained as follows:

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
```

However, that requires its base to have `operator-` defined. We should change the constraint to `sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>`:

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;
```

[2020-11-01; Daniel comments]

Bullet (2) of the discussion has already been resolved by LWG [3483](#), it has therefore been omitted from the proposed wording below.

[2020-11-15; Reflector prioritization]

Set priority to 0 and status to Tentatively Ready after five votes in favour during reflector discussions.

Proposed resolution:

This wording is relative to [N4868](#).

This wording intentionally only touches `operator++(int)` and *not* `operator-`, see the 2020-11-01 comment for the reason why.

1. Modify 24.7.16.3 [\[range.elements.iterator\]](#), class template `elements_view::iterator` synopsis, as indicated:

```
[...]
constexpr iterator& operator++();
constexpr void operator++(int) requires (!forward_range<Base>);;
constexpr iterator operator++(int) requires forward_range<Base>;
[...]
```

[...]

```
constexpr void operator++(int) requires (!forward_range<Base>);;
```

-6- *Effects:* Equivalent to: `++current_.`

```
constexpr iterator operator++(int) requires forward_range<Base>;
```

[...]

[3494](#). Allow ranges to be conditionally borrowed

Section: 24.7.15 [\[range.reverse\]](#), 24.7.7 [\[range.take\]](#), 24.7.9 [\[range.drop\]](#), 24.7.10 [\[range.drop.while\]](#), 24.7.14 [\[range.common\]](#), 24.7.10 [\[range.drop.while\]](#), 24.7.16 [\[range.elements\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Barry Revzin **Opened:** 2020-11-01 **Last modified:** 2021-01-15

Priority: Not Prioritized

Discussion:

Consider the following approach to trimming a `std::string`:

```
auto trim(std::string const& s) {
    auto isalpha = [](unsigned char c){ return std::isalpha(c); };
    auto b = ranges::find_if(s, isalpha);
    auto e = ranges::find_if(s | views::reverse, isalpha).base();
    return subrange(b, e);
}
```

This is a fairly nice and, importantly, safe way to implement `trim`. The iterators `b` and `e` returned from `find_if` will not dangle, since they point into the string `s` whose lifetime outlives the function. But the status quo in C++20 is that `s | views::reverse` is not a borrowed range (because `reverse_view<V>` is never a borrowed range for any `V`). As a result, `find_if(s | views::reverse, isalpha)` returns *dangling* rather than a real iterator.

Instead, you have to write it this way, introducing a new named variable for the reversed view:

```
auto trim(std::string const& s) {
    auto isalpha = [](unsigned char c){ return std::isalpha(c); };
    auto b = ranges::find_if(s, isalpha);
    auto reversed = s | views::reverse;
    auto e = ranges::find_if(reversed, isalpha).base();
    return subrange(b, e);
}
```

But borrowed range can be a transitive property. `s` itself is a borrowed range (as all lvalue references are) so `s | views::reverse` could be made to be too, which would allow the first example above to work with really no downside. We know such an iterator would not dangle, we just need to teach the library this.

[P2017R1](#) resolves this by making `reverse_view<V>` a borrowed range when `V` is a borrowed range (and likewise several other range adapters).

Rationale:

Resolved by [P2017R1](#).

[2021-01-15; Telecon prioritization]

Set status to Tentatively Ready after five P0 votes in reflector discussion.

Proposed resolution:

[3495](#). `constexpr` `launder` makes pointers to inactive members of unions usable

Section: 17.6.5 [\[ptr.launder\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hubert Tong **Opened:** 2020-11-10 **Last modified:** 2021-02-08

Priority: 3

View all other [issues](#) in `[ptr.launder]`.

Discussion:

The wording in 17.6.5 [\[ptr.launder\]](#) paragraph 4:

An invocation of this function may be used in a core constant expression whenever the value of its argument may be used in a core constant expression.

can be taken to mean that the invocation may be used only when the value of its argument can be used in place of the invocation itself.

That interpretation is not particularly obvious, but based on comments on the CWG reflector (see [here](#)), that is the interpretation that matches the design intent.

Consider:

```
#include <new>

struct A { int x; int y; };
struct B { float x; int y; };

union U {
    A a;
    B b;
};

constexpr A foo() {
    U u;
    int* byp = &u.b.y;
    static_assert(&u.b.y == static_cast<void*>(&u.a.y));
    u.a.y = 42;
    *std::launder(byp) = 13;
    return u.a;
}

extern constexpr A globA = foo();
```

If the `static_assert` succeeds, then a possible interpretation is that the source file above compiles because the call to `std::launder` produces a pointer to `u.a.y`. That interpretation is apparently not desirable.

[2020-11-21; Reflector prioritization]

Set priority to 3 during reflector discussions.

[2020-12-07; Davis Herring comments]

This issue is related to [CWG 2464](#).

[2021-02-08; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4868](#).

1. Modify 17.6.5 [[ptr.launder](#)] as indicated:

```
template<class T> [[nodiscard]] constexpr T* launder(T* p) noexcept;
```

[...]

-4- *Remarks:* An invocation of this function may be used in a core constant expression ~~whenever the~~ if and only if the (converted) value of its argument may be used in ~~a core constant expression~~ place of the function invocation. A byte of storage *b* is reachable through a pointer value that points to an object *Y* if there is an object *Z*, pointer-interconvertible with *Y*, such that *b* is within the storage occupied by *Z*, or the immediately-enclosing array object if *Z* is an array element.

[...]

[3500](#). `join_view::iterator::operator->()` is bogus

Section: 24.7.11.3 [[range.join.iterator](#)] **Status:** [Tentatively Ready](#) **Submitter:** Michael Schellenberger Costa **Opened:** 2020-11-15
Last modified: 2020-11-21

Priority: 0

View other [active issues](#) in [range.join.iterator].

View all other [issues](#) in [range.join.iterator].

Discussion:

There seems to be a copy & paste error in the `join_view::iterator::operator->()` specification. According to 24.7.11.3 [\[range.join.iterator\]](#) p8 it is specified as:

```
constexpr iterator_t<Base> operator->() const
    requires has_arrow<iterator_t<Base>> && copyable<iterator_t<Base>>;
```

-8- *Effects:* Equivalent to return `inner_;`

Now `inner_` is of type `iterator_t<range_reference_t<Base>>`. So it is unclear how that should be converted to `iterator_t<Base>`, or why the constraints concern the outer iterator type `iterator_t<Base>`. On the other hand returning `outer_` would not make any sense here.

As far as I can tell we should replace `iterator_t<Base>` with `iterator_t<range_reference_t<Base>>` so that the new specification would read

```
constexpr iterator_t<range_reference_t<Base>> operator->() const
    requires has_arrow<iterator_t<range_reference_t<Base>>>
    && copyable<iterator_t<range_reference_t<Base>>>;
```

-8- *Effects:* Equivalent to return `inner_;`

Generally it would help readability of the specification a lot if we would introduce some exposition only aliases:

```
using OuterIter = iterator_t<Base>; //exposition-only
using InnerIter = iterator_t<range_reference_t<Base>> //exposition-only
```

and use them throughout `join_view::iterator`.

[2020-11-21; Reflector prioritization]

Set priority to 0 and status to Tentatively Ready after six votes in favour during reflector discussions.

Proposed resolution:

This wording is relative to [N4868](#).

1. Modify 24.7.11.3 [\[range.join.iterator\]](#), class template `join_view::iterator` synopsis, as indicated:

```
template<input_range V>
    requires view<V> && input_range<range_reference_t<V>> &&
        (is_reference_v<range_reference_t<V>> ||
         view<range_value_t<V>>)
struct join_view<V>::iterator {
private:
    using Parent = // exposition only
        conditional_t<Const, const join_view, join_view>;
    using Base = conditional_t<Const, const V, V>; // exposition only
    using OuterIter = iterator_t<Base>; //exposition-only
    using InnerIter = iterator_t<range_reference_t<Base>> //exposition-only
    static constexpr bool ref-is-glvalue = // exposition only
        is_reference_v<range_reference_t<Base>>;
    OuterIter iterator_t<Base> outer_ = OuterIter iterator_t<Base>(); // exposition only
    InnerIter iterator_t<range_reference_t<Base>> inner_ = // exposition only
        InnerIter iterator_t<range_reference_t<Base>>();
    Parent* parent_ = nullptr; // exposition only
    constexpr void satisfy(); // exposition only
public:
    [...]
    iterator() = default;
    constexpr iterator(Parent& parent, OuterIter iterator_t<Base> outer);
    constexpr iterator(iterator<!Const> i)
        requires Const &&
            convertible_to<iterator_t<V>, OuterIter iterator_t<Base>> &&
            convertible_to<iterator_t<InnerRng>,
```

```

        InnerIteriterator_t<range_reference_t<Base>>>;

constexpr decltype(auto) operator*() const { return *inner_; }

constexpr InnerIteriterator_t<Base> operator->() const
    requires has_arrow<InnerIteriterator_t<Base>>
    && copyable<InnerIteriterator_t<Base>>;

constexpr iterator& operator++();
[...]

[...]

constexpr void satisfy(); // exposition only

-5- Effects: Equivalent to:

    [...]
    if constexpr (ref-is-glvalue)
        inner_ = InnerIteriterator_t<range_reference_t<Base>>>();

constexpr iterator(Parent& parent, OuterIteriterator_t<Base> outer);

[...]

constexpr iterator(iterator<!Const> i)
    requires Const &&
        convertible_to<iterator_t<V>, OuterIteriterator_t<Base>> &&
        convertible_to<iterator_t<InnerRng>,
            InnerIteriterator_t<range_reference_t<Base>>>;

[...]

constexpr InnerIteriterator_t<Base> operator->() const
    requires has_arrow<InnerIteriterator_t<Base>>
    && copyable<InnerIteriterator_t<Base>>;

-8- Effects: Equivalent to return inner_;

```

3502. elements_view should not be allowed to return dangling references

Section: 24.7.16.3 [\[range.elements.iterator\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2020-11-18 **Last modified:** 2021-02-08

Priority: 2

View other [active issues](#) in [\[range.elements.iterator\]](#).

View all other [issues](#) in [\[range.elements.iterator\]](#).

Discussion:

This compiles but the resulting view is full of dangling references:

```

std::vector<int> vec = {42};
auto r = vec | std::views::transform([](auto c) { return std::make_tuple(c, c); })
    | std::views::keys;

```

This is because `elements_view::iterator::operator*` is specified as

```

constexpr decltype(auto) operator*() const { return get<N>(*current_); }

```

Here `*current_` is a prvalue, and so the `get<N>` produces a reference into the materialized temporary that becomes dangling as soon as `operator*` returns.

We should either ban this case altogether, or make `operator*` (and `operator[]`) return by value when `*current_` is a prvalue and the corresponding tuple element is not a reference (since this `get` is `std::get`, we need not worry about weird user-defined overloads.)

[2020-11-29; Reflector prioritization]

Set priority to 2 during reflector discussions.

[2021-01-31 Tim adds PR]

[2021-02-08; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4878](#).

1. Modify 24.7.16.2 [\[range.elements.view\]](#), as indicated:

```
namespace std::ranges {
    template<class T, size_t N>
        concept has-tuple-element =                // exposition only
            requires(T t) {
                typename tuple_size<T>::type;
                requires N < tuple_size_v<T>;
                typename tuple_element_t<N, T>;
                { get<N>(t) } -> convertible_to<const tuple_element_t<N, T>&>;
            };

    template<class T, size_t N>
        concept returnable-element = is_reference_v<T> || move_constructible<tuple_element_t<N, T>>;

    template<input_range V, size_t N>
        requires view<V> && has-tuple-element<range_value_t<V>, N> &&
            has-tuple-element<remove_reference_t<range_reference_t<V>>, N> &&
            returnable-element<range_reference_t<V>, N>
        class elements_view : public view_interface<elements_view<V, N>> {
        [...]
        };
}
```

2. Modify 24.7.16.3 [\[range.elements.iterator\]](#) as indicated:

```
namespace std::ranges {
    template<input_range V, size_t N>
        requires view<V> && has-tuple-element<range_value_t<V>, N> &&
            has-tuple-element<remove_reference_t<range_reference_t<V>>, N> &&
            returnable-element<range_reference_t<V>, N>
    template<bool Const>
        class elements_view<V, N>::iterator {                // exposition only
            using Base = maybe_const<Const, V>;              // exposition only

            iterator_t<Base> current_ = iterator_t<Base>();    // exposition only

            static constexpr decltype(auto) get-element(const iterator t<Base>& i); // exposition only
        public:
            [...]
            constexpr decltype(auto) operator*() const
            { return get<N>get-element(*current_); }

            [...]
            constexpr decltype(auto) operator[](difference_type n) const
            requires random_access_range<Base>
            { return get<N>get-element(*current_ + n); }
        };
}

static constexpr decltype(auto) get-element(const iterator t<Base>& i); // exposition only
```

[-? Effects: Equivalent to:

```
if constexpr (is_reference_v<range_reference_t<Base>>)_ {
    return get<N>(*i);
}
else {
    using E = remove_cv_t<tuple_element_t<N, range_reference_t<Base>>>;
    return static_cast<E>(get<N>(*i));
}
```

3. Modify 24.7.16.4 [\[range.elements.sentinel\]](#) as indicated:

```
namespace std::ranges {
    template<input_range V, size_t N>
        requires view<V> && has_tuple_element<range_value_t<V>, N> &&
            has_tuple_element<remove_reference_t<range_reference_t<V>>, N> &&
                returnable_element<range_reference_t<V>, N>
        template<bool Const>
        class elements_view<V, N>::sentinel { // exposition only
        [...];
        };
}
```

3505. split_view::outer-iterator::operator++ misspecified

Section: 24.7.12.3 [\[range.split.outer\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2020-11-20 **Last modified:** 2021-02-08

Priority: 2

View all other [issues](#) in [\[range.split.outer\]](#).

Discussion:

Prior to the application of [P1862R1](#), the part of `split_view::outer-iterator::operator++` that searches for the pattern is specified as:

```
do {
    const auto [b, p] = ranges::mismatch(current, end, pbegin, pend);
    if (p == pend) {
        current = b; // The pattern matched; skip it
        break;
    }
} while (++current != end);
```

[P1862R1](#), trying to accommodate move-only iterators, respecified this as

```
do {
    auto [b, p] = ranges::mismatch(std::move(current), end, pbegin, pend);
    current = std::move(b);
    if (p == pend) {
        break; // The pattern matched; skip it
    }
} while (++current != end);
```

but this is not correct, because if the pattern didn't match, it advances `current` to the point of first mismatch, skipping elements before that point. This is totally wrong if the pattern contains more than one element.

Consider `std::views::split("xxyx"sv, "xy"sv)`:

- at the beginning, `current` points to the first 'x'
- `ranges::mismatch` produces [iterator to second 'x', iterator to 'y' in pattern]
- `current` now points to second 'x'
- we increment `current` in the condition, so it now points to 'y'

At this point there's no way we can find the "xy" in the middle. In fact, in this particular example, we'll increment past the end of the source range at the end of the third iteration.

[2020-11-29; Reflector prioritization]

Set priority to 2 during reflector discussions.

[2021-02-08; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4868](#).

1. Modify 24.7.12.3 [\[range.split.outer\]](#) as indicated:

```
constexpr outer-iterator& operator++();
```

-6- *Effects*: Equivalent to:

```
const auto end = ranges::end(parent_>base_);
if (current == end) return *this;
const auto [pbegin, pend] = subrange{parent_>pattern_};
if (pbegin == pend) ++current;
else if constexpr (tiny-range<Pattern>) {
    current = ranges::find(std::move(current), end, *pbegin);
    if (current != end) {
        ++current;
    }
}
else {
    do {
        auto [b, p] = ranges::mismatch(std::move(current), end, pbegin, pend);
        current = std::move(b);
        if (p == pend) {
            current = b;
            break; // The pattern matched; skip it
        }
    } while (++current != end);
}
return *this;
```